SDS – ESPECIFICACION DE DISEÑO DE SOFTWARE

1. Generalidades del Documento

1.1 Objetivo del Documento

Definir de manera detallada y estructurada la arquitectura del sistema DieselMaps, especificando sus componentes internos, relaciones, interfaces, flujos de información y modelo de datos, con el fin de consolidar una solución técnica coherente y alineada con los requerimientos definidos en fases previas.

Asimismo, documentar y justificar las decisiones de arquitectura y diseño adoptadas, considerando atributos de calidad como disponibilidad, seguridad, integridad y rendimiento, garantizando la trazabilidad entre los requerimientos y la solución propuesta.

Adicionalmente, refinar y ampliar las vistas arquitectónicas (contexto, funcional, información, despliegue, desarrollo, procesos y operacional), incorporando un mayor nivel de detalle técnico que permita comprender el comportamiento del sistema, su distribución, sus mecanismos de soporte y sus estrategias de continuidad y resiliencia.

Finalmente, establecer las bases técnicas necesarias para la implementación del sistema, facilitando su desarrollo, mantenimiento, validación y evolución en etapas posteriores del ciclo de vida del software.

1.2 Audiencia del Documento

Este documento está dirigido a los diferentes actores involucrados en el diseño, desarrollo, validación y operación del sistema DieselMaps, incluyendo:

- **Equipo de desarrollo:** responsables de la implementación del sistema, quienes utilizarán este documento como guía para comprender la arquitectura, los componentes, las interfaces y las decisiones técnicas adoptadas.
- **Arquitecto de software:** encargado de definir, validar y evolucionar la arquitectura del sistema, asegurando el cumplimiento de los atributos de calidad y la coherencia técnica de la solución.
- **Equipo de pruebas (QA):** quienes emplearán este documento para diseñar casos de prueba basados en la arquitectura, validar el comportamiento del sistema y verificar requisitos funcionales y no funcionales.
- **Stakeholders técnicos:** interesados en comprender la solución desde una perspectiva técnica, incluyendo líderes de proyecto, revisores académicos o evaluadores.
- **Administradores del sistema (DevOps/operación):** responsables del despliegue, monitoreo, mantenimiento y continuidad del sistema en ambientes productivos.

1.3 Acrónimos

Acrónimo	Significado
SDS	Software Design Specification

Acrónimo	Significado
UML	Unified Modeling Language
API	Application Programming Interface
BD	Base de Datos
ER	Entidad-Relación

2. Introducción

2.1 Descripción del Problema

En la actualidad, los conductores no cuentan con una herramienta centralizada que les permita consultar de manera rápida, confiable y en tiempo cercano al real los precios del combustible, comparar estaciones de servicio cercanas, calcular rutas óptimas y estimar el costo total de un viaje.

Esta falta de integración de información genera dificultades en la toma de decisiones, incrementa los gastos en combustible y limita la optimización de tiempo y recursos.

Adicionalmente, la dependencia de múltiples fuentes externas de información introduce problemas de disponibilidad, consistencia y confiabilidad de los datos.

2.2 Objetivos de la Aplicación

Desarrollar una aplicación que permita:

- Consultar estaciones de servicio cercanas
- Comparar precios de combustible
- Visualizar rutas óptimas
- Simular costos de viaje
- Calcular ahorro estimado

2.3 Restricciones

El sistema debe considerar las siguientes restricciones:

- **Compatibilidad tecnológica:**
  Debe funcionar en los dispositivos y entornos disponibles para el equipo de desarrollo.
- **Acceso a datos:**
  Existe una limitación en la disponibilidad de datos reales en tiempo real sobre precios de combustible.
- **Infraestructura:**
  El uso de servicios en la nube debe optimizarse para evitar sobrecostos en almacenamiento y procesamiento.

- **Recursos del equipo:**

  El desarrollo debe realizarse utilizando herramientas y tecnologías disponibles para el equipo.

- **Dependencia de servicios externos:**

  Acceso limitado o restringido a APIs comerciales o servicios externos.

- **Arquitectura existente:**

  Se trabaja bajo una arquitectura monolítica que limita la escalabilidad inmediata.

3. Arquitectura de Software

3.1 Estilo de Arquitectura

Se tomó la decisión de adoptar una arquitectura **monolítica por capas**, considerando, el tamaño del equipo y el alcance funcional definido.

La selección de este estilo arquitectónico responde a la necesidad de mantener una estructura clara, organizada y controlada, mediante la separación en capas de presentación, lógica de negocio y persistencia. Este enfoque permite una adecuada distribución de responsabilidades, facilitando el desarrollo, mantenimiento y evolución del sistema.

El uso de tecnologías como React para la capa de presentación y Spring Boot para la lógica de negocio y acceso a datos, refuerza la adopción de una arquitectura por capas, al permitir una integración estructurada a través de servicios REST.

Desde la perspectiva del equipo de arquitectura, esta decisión prioriza la gestión de la complejidad, la eficiencia en el desarrollo y la mantenibilidad, evitando la sobrecarga técnica asociada a arquitecturas distribuidas en una etapa temprana del proyecto.

Justificación de la elección:

- **Desarrollo unificado:**
  Todo el sistema se gestiona en una única base de código, facilitando el desarrollo y mantenimiento inicial.
- **Alto rendimiento:**
  La comunicación entre componentes es directa (en memoria), reduciendo latencias.
- **Simplicidad:**
  Menor complejidad en comparación con arquitecturas distribuidas.
- **Despliegue simple:**
  La aplicación se despliega como una sola unidad.

4. Decisiones de Arquitectura (SDS)

A continuación, se presentan las principales decisiones adoptadas:

1. Manejo de disponibilidad del sistema

Se implementarán mecanismos de respaldo (backups) y opciones de despliegue alternativo (nube o servidor secundario) para garantizar la continuidad del servicio ante fallos.

2. Gestión de servicios externos

Se utilizarán estrategias de **caché y fallback** para reducir la dependencia de APIs externas (como mapas), permitiendo que el sistema funcione en modo degradado si estas fallan.

3. Seguridad de credenciales

Las credenciales, claves y configuraciones sensibles se gestionarán mediante **variables de entorno**, evitando su exposición en el código fuente.

4. Integridad de datos

Se aplicarán validaciones antes del almacenamiento y uso de transacciones en la base de datos para asegurar la consistencia de la información.

5. Control de acceso

Se implementará un sistema de autenticación y autorización basado en roles para restringir el acceso a funcionalidades críticas.

6. Protección ante ataques

Se aplicarán medidas de seguridad como:

- Validación de entradas
- Prevención de inyección SQL
- Manejo seguro de formularios

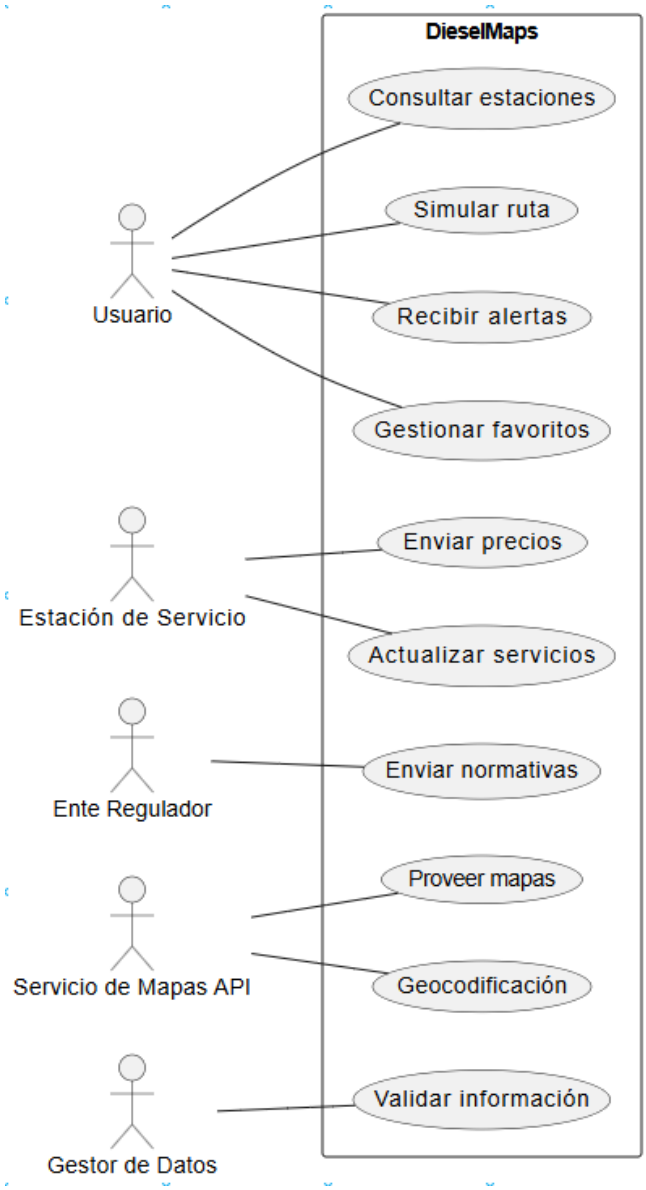
7. Escalabilidad futura

Aunque el sistema es monolítico, se diseñará de forma modular para permitir una posible migración a microservicios en el futuro.

3.2 Vistas de Arquitectura

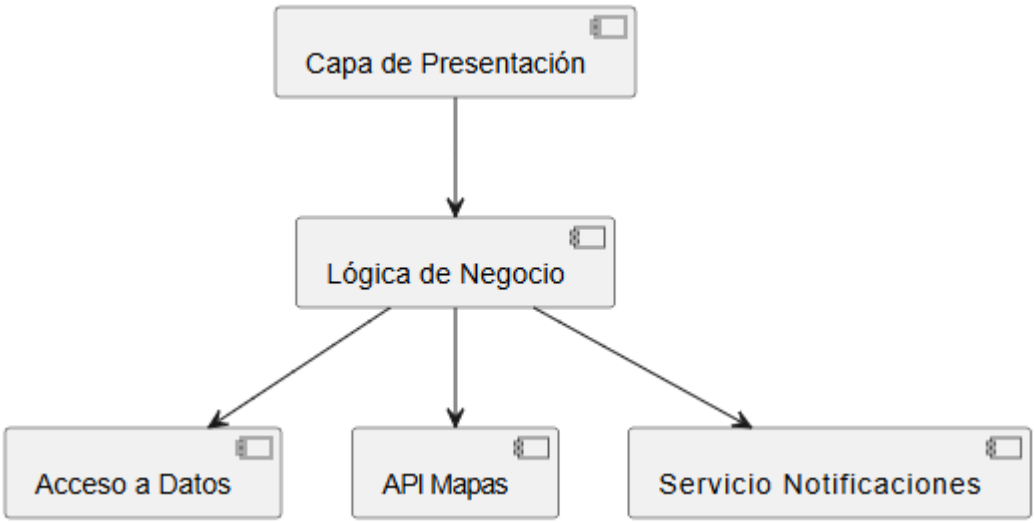
3.2.1 Vista de Contexto (UML – Casos de Uso)

Este diagrama sitúa al sistema DieselMaps dentro de su entorno, identificando los actores externos que interactúan con él y los flujos de información que los conectan. El sistema recibe precios de combustible desde las Estaciones de Servicio (EDS), normativas de precios máximos desde el Ente Regulador, y coordenadas y umbrales de ahorro desde el Usuario Conductor. A su vez, emite alertas de precio cuando se cumple la condición configurada. La integración con la API de Mapas provee la capa geográfica necesaria para el cálculo y visualización de rutas. El Gestor de Datos y Normativa actúa como actor administrativo, validando que la información ingresada por las estaciones cumpla la regulación vigente. Los casos de uso se distribuyen en dos ciclos: el Ciclo 1 cubre las funcionalidades base (registro, autenticación, consulta de estaciones, cálculo de rutas y simulación de costos), mientras que el Ciclo 2 incorpora funcionalidades de valor añadido como favoritos, notificaciones automáticas de cambio de precio y recomendaciones de estaciones económicas.



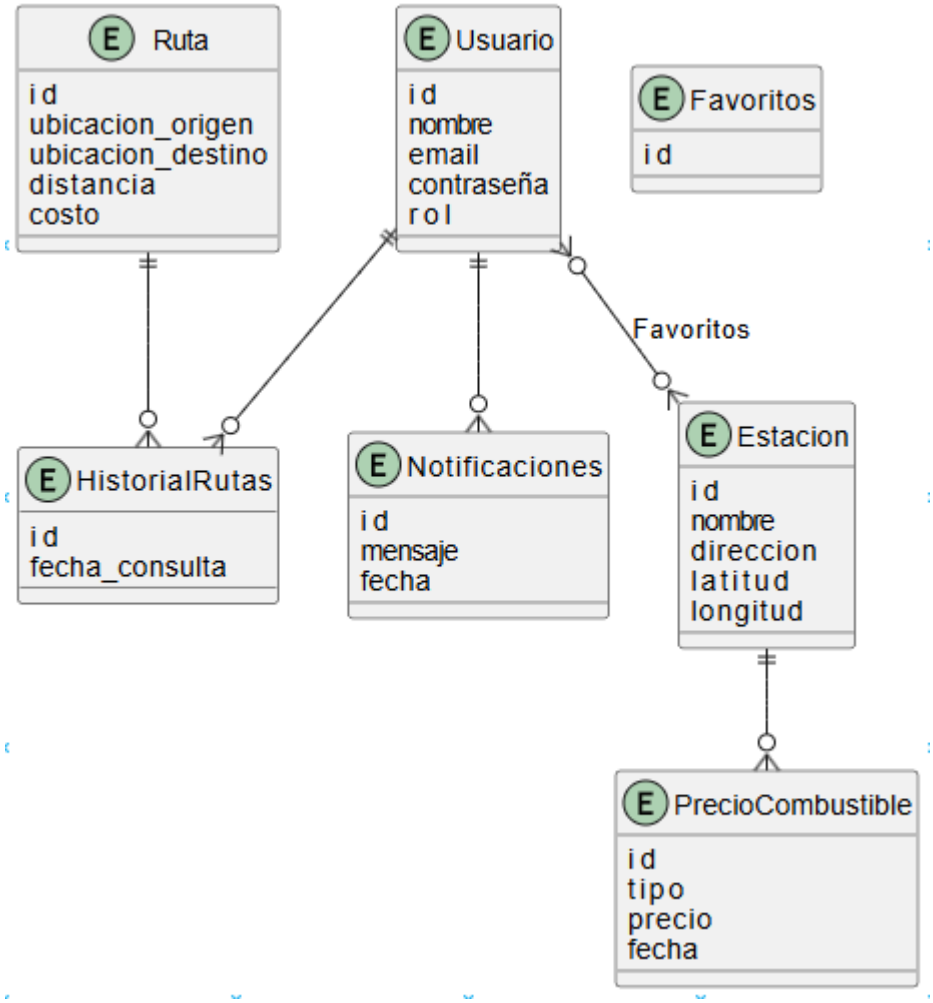
3.2.2 Vista Funcional (UML – Componentes)

El sistema adopta una arquitectura monolítica organizada en tres capas internas con responsabilidades bien delimitadas. La Capa de Presentación expone los controladores REST que reciben las solicitudes del cliente y devuelven respuestas estructuradas en JSON. La Capa de Lógica de Negocio concentra las reglas del dominio: validación de datos, cálculo de ahorro, comparación de precios y gestión de autenticación mediante JWT. La Capa de Acceso a Datos encapsula todas las operaciones sobre la base de datos SQLite, garantizando la integridad referencial y el aislamiento de las consultas. El módulo de Seguridad es transversal: actúa sobre la capa de presentación para filtrar accesos no autorizados y sobre la capa de servicio para controlar permisos por rol (Administrador / Conductor). Las dependencias fluyen en una sola dirección: Presentación → Servicio → Repositorio → Modelo, lo que facilita el mantenimiento y las pruebas unitarias por capa.



3.2.3 Vista de Información (Modelo Entidad-Relación)

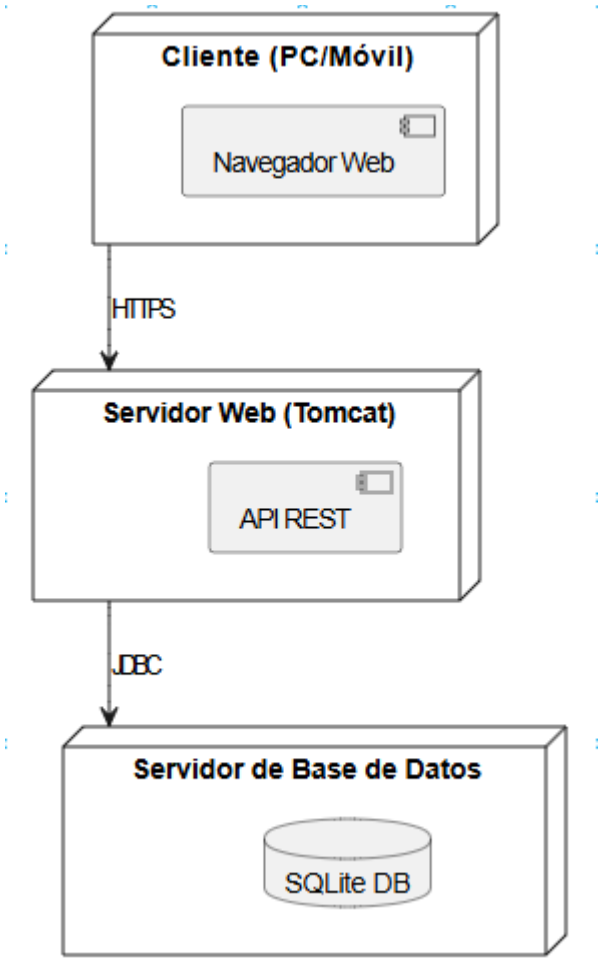
El modelo de datos refleja las entidades centrales del dominio y sus relaciones. La entidad Usuario almacena credenciales y rol de acceso. Estación representa cada punto de servicio con su ubicación geográfica y horario. Ruta asocia dos registros de Ubicación (origen y destino) y almacena la distancia y el costo calculado. PrecioCombustible registra el histórico de precios por tipo de combustible para cada estación, lo que permite comparaciones en el tiempo. La tabla Favoritos actúa como tabla pivot para la relación muchos-a-muchos entre Usuario y Estación. Historial Rutas vincula usuarios con rutas consultadas, conservando la fecha de consulta para análisis posterior. Finalmente, Notificaciones registra cada alerta emitida al usuario, con su tipo, contenido y estado de lectura. Las cardinalidades garantizan que un usuario pueda tener múltiples rutas en su historial, múltiples estaciones favoritas y múltiples notificaciones, mientras que cada estación puede acumular múltiples registros de precio a lo largo del tiempo.



3.2.4 Vista de Despliegue (UML – Despliegue)

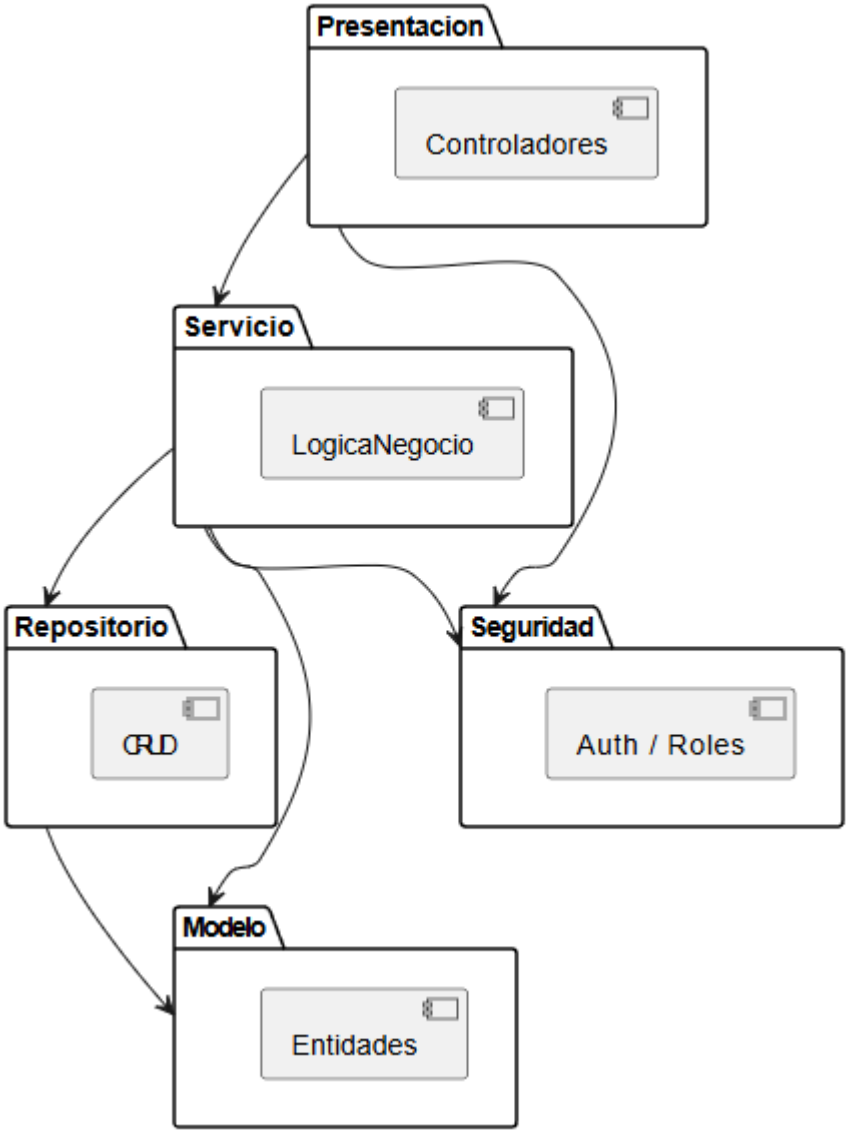
La infraestructura de DieselMaps se distribuye en tres nodos principales. El Cliente, que puede ser un navegador web o una aplicación móvil, es el punto de entrada del usuario y se comunica con el servidor mediante HTTPS, garantizando la confidencialidad de los datos en tránsito. El Servidor de Aplicaciones, ejecutado sobre Apache Tomcat, aloja la API REST y procesa toda la lógica de negocio asociada a los casos de uso UC-001 a UC-018. Se comunica con el cliente mediante

HTTP/REST y con la base de datos mediante JDBC. El Servidor de Base de Datos aloja la instancia SQLite que persiste usuarios, estaciones, rutas, precios e historial. El flujo completo de comunicación sigue la cadena: Cliente → HTTPS → Servidor Web → REST/HTTP → Servidor de Aplicaciones → JDBC → Base de Datos, lo que delimita claramente las responsabilidades de cada nodo y los protocolos utilizados en cada salto.



3.2.5 Vista de Desarrollo (UML – Paquetes)

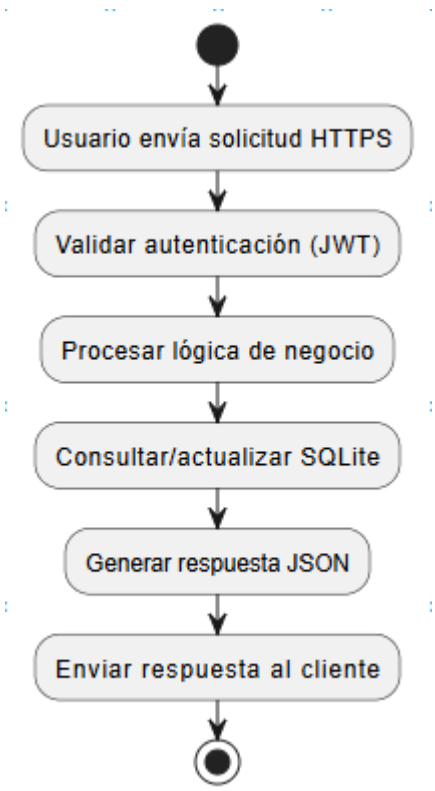
El código fuente se organiza en cinco paquetes con roles diferenciados. El paquete Presentación contiene los controladores REST que mapean los endpoints de la API a los casos de uso del sistema. El paquete Servicio centraliza la lógica de negocio, el control de transacciones y la orquestación entre repositorios. El paquete Repositorio define las interfaces de acceso a datos con operaciones CRUD y consultas especializadas sobre SQLite. El paquete Modelo agrupa las entidades del dominio con sus atributos y comportamientos. El paquete Seguridad implementa la autenticación JWT y la gestión de roles, siendo utilizado de forma transversal por los paquetes de Presentación y Servicio. Las dependencias siguen una dirección clara y acíclica: Presentación depende de Servicio, Servicio depende de Repositorio y Modelo, y Repositorio depende únicamente de Modelo. Esta estructura facilita la incorporación de nuevas funcionalidades sin afectar capas superiores.



3.2.6 Vista de procesos (UML – Procesos)

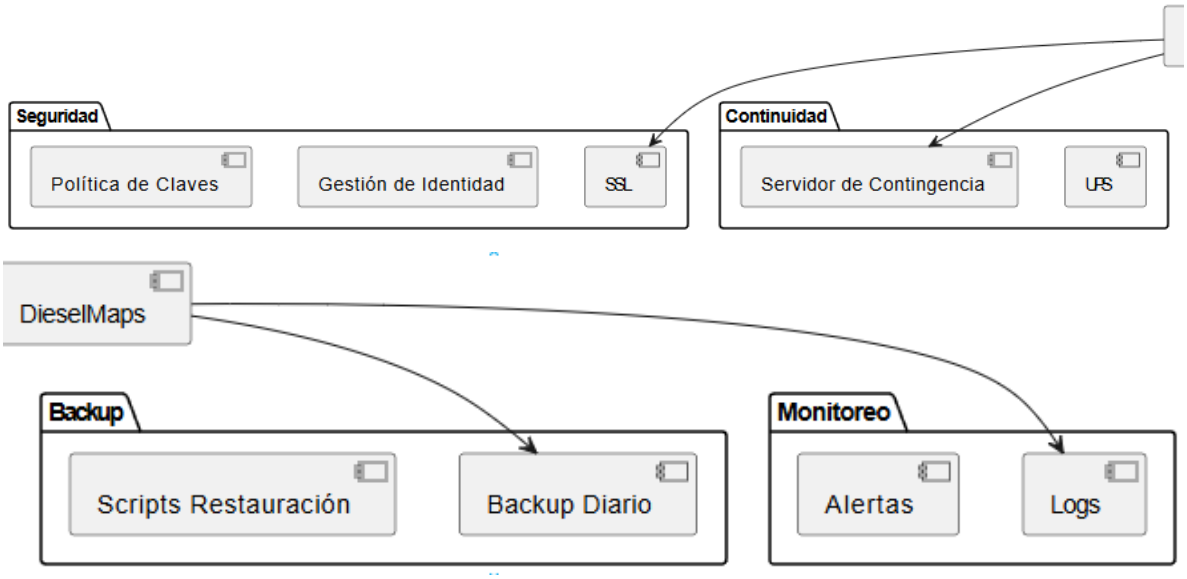
Esta vista describe el comportamiento dinámico del sistema durante la ejecución. El flujo operativo estándar para un usuario inicia con el envío de una solicitud HTTPS, continúa con la validación del token JWT, el procesamiento por la capa de servicio, la

consulta o actualización en SQLite y finaliza con la respuesta JSON al cliente. El flujo del administrador sigue la misma cadena, pero accede al panel de gestión, donde puede ejecutar operaciones CRUD sobre estaciones y precios, quedando cada acción registrada en el log de auditoría. En términos de concurrencia, el caso de uso UC-017 (filtrar estaciones por servicios) es representativo: múltiples usuarios pueden realizar solicitudes simultáneas, las cuales son atendidas en hilos independientes gestionados por el contenedor Tomcat. El pool de conexiones JDBC evita bloqueos sobre SQLite y asegura que la concurrencia no comprometa la integridad de los datos ni el tiempo de respuesta del sistema.



3.2.7 Vista Operacional (UML – Operacional)

Esta vista describe las políticas y mecanismos de soporte que garantizan la operación continua y segura de DieselMaps, complementando las vistas funcionales con una perspectiva de infraestructura y gobierno. En materia de seguridad e identidad, el sistema utiliza certificados SSL para cifrar toda la comunicación y establece una política de rotación de claves cada 90 días para los usuarios administradores. El firewall restringe el acceso a correo personal desde el entorno de administración, y la política de endpoint security impide el uso de memorias USB para prevenir la exfiltración de datos sensibles. La continuidad del servicio se apoya en un sistema UPS que protege el servidor de base de datos ante cortes eléctricos y en un servidor de contingencia que activa el modo failover automáticamente ante la caída del nodo principal. El esquema de backup contempla respaldos diarios automatizados con scripts de restauración validados. El monitoreo registra de forma continua los intentos de inicio de sesión, cambios de precio, eliminaciones de estaciones y errores críticos del sistema, permitiendo tanto la auditoría administrativa como la detección temprana de actividad anómala.



3.3 Decisiones de Arquitectura

Las decisiones adoptadas responden directamente a los requerimientos arquitecturalmente significativos del sistema, en especial aquellos relacionados con **seguridad, integridad de datos y disponibilidad**, que son críticos para el funcionamiento de DieselMaps.

En el caso de la **validación de precios frente a la normativa**, la principal preocupación es garantizar que la información proporcionada por las estaciones de servicio sea confiable y cumpla con las regulaciones del ente regulador. Para ello, se evaluaron alternativas como validar en la base de datos, validar en la capa de presentación o validar en la lógica de negocio. Se seleccionó la validación en la **capa de lógica de negocio**, ya que permite centralizar las reglas, asegurar consistencia antes del almacenamiento y mantener bajo acoplamiento entre capas, cumpliendo criterios de integridad, mantenibilidad y trazabilidad.

Por otro lado, respecto a la **protección de la comunicación entre cliente, servidor y servicios externos**, la preocupación principal es evitar interceptación o manipulación de datos sensibles (coordenadas, precios, autenticación). Se consideraron alternativas como comunicación sin cifrado, cifrado parcial o cifrado completo. Se seleccionó el uso de **HTTPS con certificados SSL/TLS**, ya que garantiza confidencialidad e integridad de la información en tránsito, alineándose con los atributos de seguridad y confiabilidad del sistema, además de ser un estándar ampliamente soportado e interoperable con APIs externas.

En conjunto, estas decisiones permiten reducir riesgos asociados a datos incorrectos o vulnerabilidades de comunicación, contribuyendo a una arquitectura coherente, segura y alineada con las necesidades de los stakeholders.

1. ¿Cómo garantizar la alta disponibilidad del sistema considerando que la arquitectura es monolítica y depende de un único servidor de aplicaciones?

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/ Preocupación de diseño	Se presenta una arquitectura monolítica desplegada en un único servidor de aplicaciones, lo que genera un punto único de fallo. En caso de caída del servidor, el sistema completo quedaría indisponible, afectando la continuidad del servicio.
Orientación de la decisión	Se busca mejorar la disponibilidad del sistema manteniendo la arquitectura monolítica, mediante estrategias de infraestructura como replicación del servidor, balanceo de carga y mecanismos de recuperación automática.

Supuestos	• El sistema está desarrollado como una aplicación monolítica. • Se dispone de infraestructura en la nube o virtualizada. • El tráfico del sistema puede crecer progresivamente.
Restricciones	• Limitación de recursos para rediseñar completamente la arquitectura. • Dependencia actual de un único despliegue del backend. • Presupuesto limitado para infraestructura avanzada.
Requerimientos relacionados	• El sistema debe ser capaz de continuar operando si una instancia del servidor falla. • Asegurar un tiempo de recuperación bajo, idealmente menor a unos minutos.
Alternativas consideradas	**Alternativa 1:** Mantener una única instancia del servidor **Alternativa 2:** Desplegar múltiples instancias (Redundancia) **Alternativa 3:** Migrar a microservicios
Evaluación de alternativas	**Alternativa 1** **Es** fácil de implementar, pero cumple requisitos de disponibilidad ya que no se puede mantener funcionando **Alternativa 2** Cumple alta disponibilidad y Permite escalar pero requiere sesiones y ñayor efuerzo de implemtnación **Alternativa 3** Alta resiliencia, escalabilidad avanzada pero no es viable por tiempo que dura sin funcionamiento y complejidad de trasladar los microservicios
Alternativa seleccionada y Justificación	alternativa 2: desplegar múltiples instancias del monolito con un balanceador de carga. Con esto elimina el punto único de fallo y puedo mantener el sistema disponible incluso si una instancia falla.

Decisiones relacionadas	Manejo de sesiones (stateless o almacenamiento compartido)
Comentarios	<Otra información no contemplada en el formato>

2. ¿Cómo asegurar la disponibilidad del sistema ante la caída de servicios externos críticos como la API de mapas?

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/ Preocupación de diseño	Dependencia de una API externa de mapas para funcionalidades críticas si este servicio falla el sistema pierde funcionalidades clave, afectando directamente la experiencia del usuario.
Orientación de la decisión	Implementar mecanismos de resiliencia para manejar fallos en la API externa, permitiendo que el sistema continúe funcionando en modo degradado.
Supuestos	• La API de mapas es un servicio externo. • El servicio puede fallar o presentar latencia. • No tengo garantías de disponibilidad del proveedor.
Restricciones	• Dependencia obligatoria del servicio externo. • No puedo modificar el comportamiento de la API externa.
Requerimientos relacionados	• El sistema **siga funcionando, aunque la API externa falle**. • El sistema debe implementar **reintentos controlados** ante fallos temporales.

	• Debo proporcionar una **respuesta alternativa cuando** la API no esté disponible. • Debo reducir la dependencia • El sistema debe registrar errores para monitoreo y análisis. •
Alternativas consideradas	**Alternativa 1:** Dependencia directa **Alternativa 2:** Implementar reintentos **Alternativa 3:** Usar múltiples APIs **Alternativa 4:** Implementar caché + fallback
Evaluación de alternativas	**Alternativa 1:** Simple y no cumple disponibilidad **Alternativa 2:** Mejora resiliencia parcial y no cubre fallos prolongados **Alternativa 3:** Alta disponibilidad más Complejidad y costo **Alternativa 4:** Alta resiliencia y mejora experiencia del usuario
Alternativa seleccionada y Justificación	Selecciono la alternativa 4: implementar caché y mecanismos de fallback. Esto me permite responder con datos previamente almacenados o degradar la funcionalidad sin interrumpir completamente el servicio.
Decisiones relacionadas	Manejo de sesiones (almacenamiento compartido) Uso de caché (memoria)

Comentarios	<Otra información no contemplada en el formato>

3. ¿Cómo evitar la pérdida de información crítica almacenada en la base de datos SQLite ante fallos del sistema o interrupciones inesperadas?

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/ Preocupación de diseño	Existe el riesgo de pérdida o corrupción de información crítica almacenada en la base de datos SQLite debido a fallos del sistema, cierres inesperados de la aplicación o interrupciones de energía.
Orientación de la decisión	Garantizar la persistencia y recuperación segura de la información almacenada en el sistema DieselMaps.
Supuestos	• El sistema utilizará SQLite como base de datos local. • El sistema puede ejecutarse en entornos con interrupciones inesperadas. • La información almacenada incluye rutas, estaciones y precios de combustible • Se requiere mantener disponibilidad y consistencia de datos.
Restricciones	• QLite no posee mecanismos avanzados de replicación como motores empresariales. • El proyecto debe mantener bajo consumo de recursos. • La solución debe ser compatible con Spring Boot.
Requerimientos relacionados	• Persistencia de información. • Disponibilidad del sistema. • Integridad de datos. • Recuperación ante fallos.

Alternativas consideradas	**Alternativa 1:** Implementar copias de seguridad automáticas periódicas de la base de datos SQLite. **Alternativa 2** Migrar a una base de datos cliente-servidor como MySQL **Alternativa 3:** Utilizar transacciones y modo WAL (Write-Ahead Logging) de SQLite.
Evaluación de alternativas	**Alternativa 1:** Fácil implementación, Bajo costo, Recuperación rápida. **Alternativa 2:** Mayor robustez y escalabilidad. **Alternativa 3:** Mejora integridad de datos.
Alternativa seleccionada y Justificación	**Alternativa 2** Implementar copias de seguridad automáticas periódicas de la base de datos SQLite, ya que esta solución permite mantener bajo costo y simplicidad, garantizando al mismo tiempo integridad y recuperación de información ante fallos inesperados.
Decisiones relacionadas	• Fácil implementación. • Bajo costo. • Recuperación rápida.
Comentarios	La solución seleccionada es adecuada para el contexto académico y tamaño actual del proyecto DieselMaps.

4. ¿Cómo garantizar la integridad de los datos de precios de combustible recibidos desde múltiples estaciones de servicio externas?

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Los precios de combustible recibidos desde diferentes estaciones pueden contener inconsistencias, datos duplicados o información incorrecta.	
Garantizar que los precios almacenados y mostrados en DieselMaps sean válidos, consistentes y confiables.	
• Las estaciones externas registran información manualmente. • Pueden existir errores humanos o datos incompletos. • El sistema almacenará históricos de precios.	
• La actualización de precios debe ser rápida. • El sistema debe soportar múltiples registros simultáneos. • Debe mantenerse compatibilidad con Spring Boot y SQLite.	
• Integridad de datos. • Consistencia de información. • Validación de entradas. • Auditoría de cambios.	
Alternativa 1: Validar precios mediante reglas de negocio antes de almacenarlos. **Alternativa 2** Permitir únicamente actualización de precios por administradores autorizados. **Alternativa 3:** Implementar historial de precios y auditoría de cambios.	
Alternativa 1: Fácil Implementación Alternativa 2: Mayor control sobre los datos. Alternativa 3: Facilita recuperación de información previa.	

Alternativa 3: Implementar historial de precios y auditoría de cambios.	
• Facilita recuperación de información previa. **Desventajas:** • Incrementa almacenamiento requerido.	
La solución permite mantener control y confiabilidad sobre los precios de combustible mostrados a los usuarios.	

5. ¿Cómo validar que los datos de precios cumplen con la normativa establecida por el ente regulador antes de ser almacenados y utilizados?

Problema/ Preocupación de diseño	¿Cómo validar que los datos de precios cumplen con la normativa establecida por el ente regulador antes de ser almacenados y utilizados?
Orientación de la decisión	Implementar un mecanismo de validación en la capa de lógica de negocio que compare los precios enviados por las estaciones de servicio con las normativas vigentes proporcionadas por el ente regulador antes de permitir su almacenamiento en la base de datos.
Supuestos	• El ente regulador provee datos actualizados de precios máximos por zona. • Las estaciones de servicio envían precios de forma periódica. • El sistema cuenta con una capa de lógica de negocio donde se pueden aplicar reglas. • Existe conectividad para obtener o actualizar la normativa.
Restricciones	• Uso de arquitectura monolítica. • Base de datos SQLite (limitaciones en validaciones complejas a nivel DB). • Dependencia de la disponibilidad de datos del ente regulador.

	• Procesamiento en tiempo real de los datos recibidos.
Requerimientos relacionados	• Validación de datos de entrada. • Integridad de la información. • Cumplimiento normativo. • Trazabilidad de cambios.
Alternativas consideradas	**Alternativa 1: Validación en la capa de presentación** **Alternativa 2: Validación en la base de datos (constraints/triggers)** **Alternativa 3: Validación en la capa lógica de negocio (backend)**
Evaluación de alternativas	**Alternativa 1: Fácil implementación - No es segura (puede ser evadida)** **Alternativa 2: Alta integridad de datos - Limitada en SQLite y difícil de mantener** **Alternativa 3: Centraliza reglas de negocio, Flexible y escalable, Permite validaciones complejas - Aumenta carga en backend**
Alternativa seleccionada y Justificación	Se selecciona la validación en la **capa de lógica de negocio**, ya que permite aplicar reglas complejas, mantener consistencia en todo el sistema y garantizar que los datos cumplan con la normativa antes de ser persistidos, sin depender de la interfaz o limitaciones del motor de base de datos.
Decisiones relacionadas	• Uso de arquitectura en capas (presentación, lógica, datos). • Implementación de validaciones de entrada. • Registro de logs para auditoría.
Comentarios	Se recomienda implementar mecanismos de actualización periódica de la normativa y manejo de inconsistencias en caso de datos desactualizados.

6. ¿Cómo proteger la comunicación entre el cliente, el servidor y los servicios externos para evitar interceptación o manipulación de datos?

Problema/ Preocupación de diseño	¿Cómo proteger la comunicación entre el cliente, el servidor y los servicios externos para evitar interceptación o manipulación de datos?
Orientación de la decisión	Implementar comunicación segura mediante HTTPS utilizando certificados SSL/TLS en todas las interacciones entre cliente-servidor y servidor-servicios externos, junto con mecanismos adicionales de autenticación y validación de integridad.
Supuestos	• El sistema será accesible a través de internet. • Existen servicios externos (API de mapas) que requieren comunicación segura. • Se dispone de certificados digitales válidos. • Los usuarios realizan autenticación en el sistema.
Restricciones	• Infraestructura limitada (arquitectura monolítica). • Dependencia de proveedores externos de API. • Costos asociados a certificados (si no se usan gratuitos). • Configuración del servidor (Tomcat).
Requerimientos relacionados	• Seguridad de la información. • Protección de datos sensibles (credenciales). • Autenticación de usuarios. • Integridad de la comunicación.
Alternativas consideradas	**Alternativa 1: Comunicación HTTP sin cifrado** **Alternativa 2: Uso de HTTPS únicamente en login** **Alternativa 3: Uso de HTTPS en toda la comunicación + autenticación (JWT)**
Evaluación de alternativas	**Alternativa 1: Simple implementación - Totalmente insegura** **Alternativa 2: Mejora parcial de seguridad - Vulnerable en otras peticiones**

	Alternativa 3: Comunicación cifrada extremo a extremo, Protección de datos sensibles, Integridad y autenticación - Requiere configuración adicional
Alternativa seleccionada y Justificación	Se selecciona el uso de **HTTPS en toda la comunicación junto con autenticación basada en tokens (JWT)**, ya que garantiza la confidencialidad, integridad y autenticidad de los datos en tránsito, cumpliendo con los requisitos de seguridad del sistema.
Decisiones relacionadas	• Implementación de autenticación JWT. • Uso de certificados SSL. • Políticas de renovación de claves. • Seguridad en endpoints REST.
Comentarios	Se recomienda complementar con medidas adicionales como validación de certificados, uso de headers de seguridad (HSTS) y protección contra ataques MITM.

7. ¿Cómo controlar el acceso al sistema para asegurar que solo usuarios autorizados puedan ejecutar operaciones según su rol?

Problema/ Preocupación de diseño	El sistema DieselMaps maneja funcionalidades que deben ser restringidas según el tipo de usuario, como la actualización de precios de combustible o la administración de estaciones. Existe el riesgo de accesos no autorizados a operaciones críticas.
Orientación de la decisión	Garantizar autenticación segura y control de acceso basado en roles para proteger las funcionalidades y datos del sistema.
Supuestos	El sistema contará con diferentes tipos de usuarios (usuario, operador y administrador). Algunas operaciones requieren privilegios especiales. El backend será desarrollado con Spring Boot. El sistema utilizará autenticación mediante credenciales
Restricciones	La solución debe ser compatible con Spring Security. El control de acceso debe ser escalable para futuras funcionalidades. Debe mantenerse un buen rendimiento del sistema.
Requerimientos relacionados	Seguridad del sistema. Autenticación de usuarios. Control de acceso por roles. Protección de operaciones críticas.
Alternativas consideradas	**Alternativa 1** Implementar autenticación básica mediante usuario y contraseña. **Alternativa 2** Implementar control de acceso basado en roles (RBAC). **Alternativa 3** Utilizar autenticación con tokens JWT.
Evaluación de alternativas	**Alternativa 1** Fácil implementación, Bajo nivel de complejidad. /Seguridad limitada, Difícil control de permisos **Alternativa 2** Restringir funcionalidades según rol, Escalable y organizado/Requiere configuración adicional **Alternativa 3** Mayor seguridad y escalabilidad, Adecuado para API REST/Mayor complejidad de implementación
Alternativa seleccionada y Justificación	-Autenticación mediante Spring Security. -Uso de JWT para sesiones seguras. -Control de acceso basado en roles (RBAC). Esta solución permite proteger las funcionalidades del sistema, restringiendo operaciones según el rol del usuario y garantizando seguridad en las peticiones realizadas al backend.

Decisiones relacionadas	Gestión de usuarios y autenticación. Seguridad del backend. Manejo de sesiones y tokens.
Comentarios	La solución propuesta facilita futuras ampliaciones del sistema y mejora la seguridad general de DieselMaps.

8. ¿Cómo prevenir ataques comunes como inyección SQL o manipulación de entradas en los formularios del sistema?

Problema/ Preocupación de diseño	El sistema DieselMaps recibe entradas de usuarios mediante formularios y consultas, lo que puede generar riesgos de seguridad como inyección SQL, manipulación de datos o ingreso de información maliciosa.
Orientación de la decisión	Garantizar la validación y protección de las entradas del sistema para evitar vulnerabilidades de seguridad.
Supuestos	Los usuarios ingresarán información mediante formularios web. El backend utilizará Spring Boot y Spring Data JPA. El sistema manejará consultas a base de datos SQLite.
Restricciones	La solución debe mantener buen rendimiento. Debe integrarse con las tecnologías actuales del proyecto. La validación debe aplicarse tanto en frontend como backend.
Requerimientos relacionados	Seguridad del sistema. Integridad de datos. Validación de entradas. Protección de base de datos.
Alternativas consideradas	**Alternativa 1** Utilizar consultas parametrizadas mediante Spring Data JPA. **Alternativa 2** Implementar validación de entradas en frontend y backend. **Alternativa 3** Aplicar sanitización y filtrado de caracteres especiales.

Evaluación de alternativas	**Alternativa 1** Previene inyección SQL, Fácil integración con Spring Boot./ No protege otros tipos de entradas maliciosas. **Alternativa 2** Detecta datos inválidos antes de procesarlos, Mejora integridad de información./ Requiere mayor implementación. **Alternativa 3** Reduce riesgo de manipulación de datos, Mejora seguridad general./ Puede bloquear caracteres válidos si se configura incorrectamente.
Alternativa seleccionada y Justificación	-Uso de consultas parametrizadas con Spring Data JPA. -Validación de entradas en frontend y backend. -Sanitización de datos ingresados por el usuario. Esta solución permite reducir significativamente vulnerabilidades relacionadas con inyección SQL y manipulación de formularios, garantizando mayor seguridad en el sistema.
Decisiones relacionadas	Seguridad del backend. Manejo de formularios. Protección de base de datos.
Comentarios	La estrategia seleccionada sigue buenas prácticas de desarrollo seguro y es adecuada para el contexto del proyecto DieselMaps.

9. ¿Cómo garantizar la integridad de la base de datos frente a accesos concurrentes en un entorno con múltiples solicitudes simultáneas?

Problema/ Preocupación de diseño	Necesidad de registrar y reconstruir la historia de las operaciones críticas (creación, modificación, eliminación) para cumplir con estándares de seguridad, depuración de errores y auditorías legales.

Orientación de la decisión	Implementar un **Patrón de Auditoría Centralizado** mediante el uso de interceptores (Aspect Oriented Programming - AOP) y una tabla de "Logs de Auditoría" dedicada o una base de datos de series temporales.
Supuestos	Se asume que el sistema cuenta con un módulo de autenticación (JWT o similar) que provee la identidad del usuario en cada petición. Se asume que los logs deben ser inmutables.
Restricciones	El proceso de registro no debe afectar la latencia percibida por el usuario (debe ser asíncrono). Los registros de auditoría no deben almacenarse en la misma partición física que los datos operativos para evitar pérdida total en caso de falla.
Requerimientos relacionados	Seguridad; No repudio; Cumplimiento (Compliance); Mantenibilidad.
Alternativas consideradas	**Alternativa 1** Triggers de Base de Datos: Disparadores automáticos que guardan el estado anterior y nuevo en tablas de sombra. **Alternativa 2** Interceptores de Aplicación (AOP): Capturar la ejecución de métodos críticos a nivel de código. **Alternativa 3** Búsqueda de eventos: Almacenar el estado del sistema como una secuencia de eventos inmutables.
Evaluación de alternativas	**Alternativa 1** <Ventajas / Desventajas> Captura cambios fuera de la app. Desventaja: Difícil de mantener y acoplado al motor de DB. **Alternativa 2** <Ventajas / Desventajas> Fácil de implementar en frameworks modernos; permite capturar contexto de negocio. Desventaja: No detecta cambios manuales en la DB. **Alternativa n** <Ventajas / Desventajas> Trazabilidad perfecta. Desventaja: Curva de aprendizaje alta y complejidad técnica.

Alternativa seleccionada y Justificación	Alternativa 2 (AOP) combinada con persistencia asíncrona. Permite capturar quién realizó la acción y desde qué dirección IP/módulo antes de que la transacción se complete, enviando el log a un servicio independiente. Es balanceado en costo/beneficio para proyectos empresariales.
Decisiones relacionadas	Selección de herramienta de Logging (ej. ELK Stack o CloudWatch); Diseño de esquema de datos para auditoría.
Comentarios	Es vital asegurar que los logs de auditoría no contengan información sensible (passwords, PII) para cumplir con normativas de privacidad.

10. ¿Cómo asegurar la trazabilidad y auditoría de las operaciones críticas realizadas dentro del sistema?

Problema/ Preocupación de diseño	Garantizar la consistencia e integridad de los datos ante condiciones de carrera (race conditions) provocadas por múltiples solicitudes simultáneas que intentan leer y actualizar los mismos recursos de la base de datos.
Orientación de la decisión	Implementar una estrategia de Control de Concurrencia Optimista (OCC) a nivel de aplicación y base de datos mediante el uso de versiones o "timestamps". Esto afectará a la capa de persistencia y a los modelos de dominio.
Supuestos	Se asume que el sistema tiene una proporción de lectura/escritura donde las colisiones son posibles, pero no ocurren en la mayoría de las transacciones. Se asume que el motor de base de datos soporta transacciones ACID.
Restricciones	No debe degradar significativamente el rendimiento del sistema. Debe ser compatible con la arquitectura de microservicios (si aplica) y evitar bloqueos largos en las tablas.

Requerimientos relacionados	Requerimiento de Atributo de Calidad: Consistencia de datos; Disponibilidad del sistema; Rendimiento bajo carga.
Alternativas consideradas	**Alternativa 1** Bloqueo Pesimista: Bloquear el registro desde que se lee hasta que se actualiza. **Alternativa 2** Control de Concurrencia Optimista (OCC): Permitir lecturas paralelas y validar la versión al escribir. **Alternativa 3** Aislamiento de Transacciones: Configurar el nivel más alto de aislamiento en el motor SQL.
Evaluación de alternativas	**Alternativa 1** <Ventajas / Desventajas> Garantiza integridad total. Desventaja: Muy lento, causa cuellos de botella y riesgo de *deadlocks*. **Alternativa 2** <Ventajas / Desventajas> Alto rendimiento, no bloquea lecturas. Desventaja: Requiere manejar excepciones de conflicto en la aplicación. **Alternativa n** <Ventajas / Desventajas> Nativo de la DB. Desventaja: Impacto masivo en la escalabilidad.
Alternativa seleccionada y Justificación	Se añade una columna version_id a las tablas críticas. Al actualizar, se verifica que la versión no haya cambiado (WHERE id = X AND version = current_version). Es la mejor opción para sistemas web escalables ya que minimiza el tiempo de bloqueo de recursos.
Decisiones relacionadas	Política de reintentos en el cliente (Retry Policy); Manejo global de excepciones de concurrencia.
Comentarios	Para casos de alta contención (como subastas en tiempo real), se podría evaluar complementar con colas de mensajes para serializar las peticiones antes de llegar a la DB.

11. ¿Cómo garantizar la continuidad del servicio ante fallos de infraestructura como cortes de energía o fallas del servidor?

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/ Preocupación de diseño	Fallas de infraestructura como cortes de energía o caídas del servidor pueden interrumpir el servicio, generando indisponibilidad del sistema y pérdida de operaciones en curso.	
Orientación de la decisión	Implementar mecanismos de continuidad del servicio que permitan mantener la operación o recuperarla rápidamente ante fallos de infraestructura.	
Supuestos	• El sistema depende de un servidor principal. • Pueden ocurrir cortes eléctricos o fallas de hardware. • No existe alta disponibilidad actualmente. • Los usuarios requieren acceso continuo al sistema.	
Restricciones	• Presupuesto limitado para infraestructura redundante. • Arquitectura monolítica existente. • Dependencia de un único servidor en el estado actual.	
Requerimientos relacionados	• El sistema debe minimizar el tiempo de inactividad. • Debe permitir recuperación rápida ante fallos. • Debe evitar pérdida de información en interrupciones. • Debe contar con monitoreo del estado del sistema. • Debe garantizar disponibilidad básica del servicio	

Alternativas consideradas	• **Alternativa 1:** No implementar solución (dependencia total del servidor) • **Alternativa 2:** Uso de UPS (sistema de energía ininterrumpida) • **Alternativa 3:** Implementar backups automáticos y recuperación manual • **Alternativa 4:** Implementar redundancia (servidor secundario o nube)
Evaluación de alternativas	• **Alternativa 1:** Simple pero no garantiza continuidad del servicio • **Alternativa 2:** Protege ante cortes eléctricos , pero no ante fallas del servidor • **Alternativa 3:** Permite recuperar información, pero no evita la caída del servicio • **Alternativa 4:** Alta disponibilidad y continuidad, pero mayor costo y complejidad
Alternativa seleccionada y Justificación	Se selecciona la **Alternativa 4:** implementar redundancia (servidor secundario o en la nube)**.** Esto permite mantener el sistema disponible incluso si el servidor principal falla, asegurando continuidad del servicio y reduciendo tiempos de inactividad
Decisiones relacionadas	• Estrategia de backups • Uso de servicios en la nube • Monitoreo del sistema

12. ¿Cómo gestionar de forma segura las credenciales, claves de acceso y certificados utilizados por el sistema?

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/ Preocupación de diseño	La gestión insegura de credenciales, claves de acceso y certificados puede generar vulnerabilidades, accesos no autorizados y comprometer la seguridad del sistema.	
Orientación de la decisión	Implementar un manejo seguro de credenciales mediante almacenamiento cifrado y control de acceso adecuado.	
Supuestos	• El sistema utiliza credenciales para acceder a servicios externos. • Existen usuarios con distintos niveles de acceso. • Las credenciales pueden ser expuestas si no se gestionan correctamente.	
Restricciones	• Uso de tecnologías actuales como SQLite y arquitectura monolítica. • Limitaciones de conocimiento en herramientas avanzadas de seguridad. • No se cuenta con infraestructura compleja de gestión de secretos.	
Requerimientos relacionados	• Las credenciales deben almacenarse de forma segura (cifrada). • No deben exponerse en el código fuente. • Debe existir control de acceso basado en roles • Se debe proteger la comunicación con certificados (HTTPS). • Debe permitir rotación de credenciales	
Alternativas consideradas	• Alternativa 1: Guardar credenciales en texto plano • Alternativa 2: Guardar credenciales en archivos de configuración consideradas. • Alternativa 3: Uso de variables de entorno. • Alternativa 4: Uso de gestor de secretos (Vault, servicios cloud)	

Evaluación de alternativas	• Alternativa 1: Muy insegura • Alternativa 2: Mejor que texto plano, pero aún vulnerable • Alternativa 3: Buena práctica, reduce exposición y es fácil de implementar • Alternativa 4: Alta seguridad, pero mayor complejidad y costo
Alternativa seleccionada y Justificación	**Alternativa 3: uso de variables de entorno.** Permite proteger las credenciales evitando que estén en el código fuente, siendo una solución segura y fácil de implementar dentro de las limitaciones del proyecto.
Decisiones relacionadas	• Seguridad en la comunicacion (HTTPS) • Control de acceso • Gestion de configuracion

3. Diseño Detallado

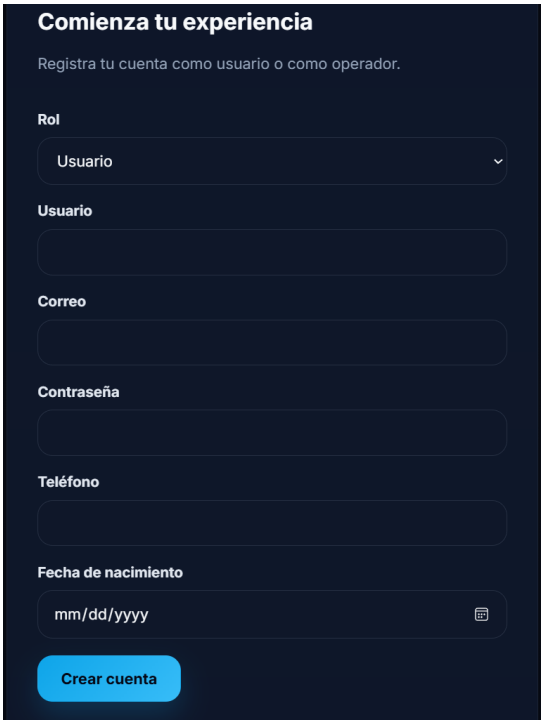
4.1 Especificación de la Interfaz de Usuario

- Mockups o prototipos
- El sistema DieselMaps contará con las siguientes pantallas principales:

1. Pantalla de Registro.
2. Pantalla de Inicio de Sesión.
3. Pantalla Principal (Mapa interactivo).
4. Pantalla de Consulta de Ubicación.
5. Panel de Administrador.
6. Pantalla de Simulación de Costo de Viaje.

- Descripción de cada pantalla

1. Pantalla de Registro de Usuario



Descripción:

Esta pantalla permite a los nuevos usuarios crear una cuenta dentro del sistema DieselMaps.
Contiene un formulario con los siguientes campos:

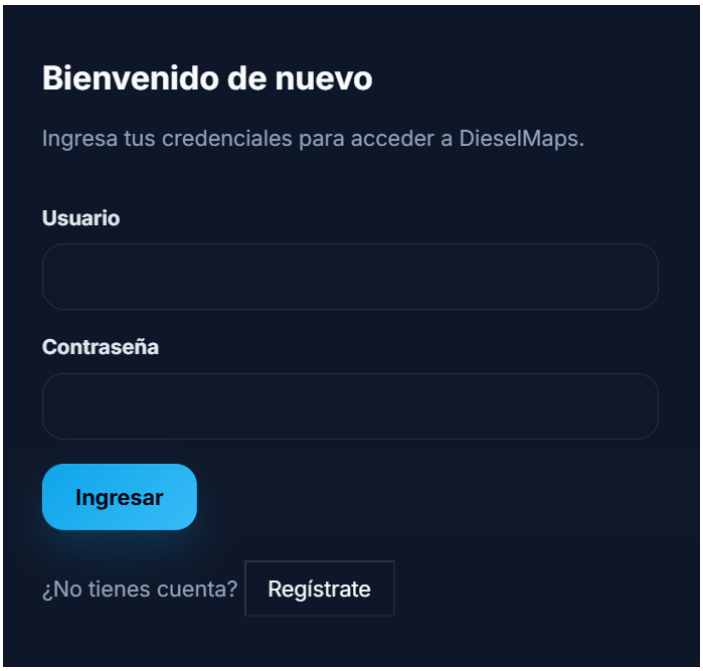
- Nombre completo
- Rol
- Correo electrónico
- Contraseña
- Teléfono
- Fecha de nacimiento

Incluye un botón “Registrarse” y un enlace para ir a la pantalla de inicio de sesión.

Comportamiento técnico

- Validación con react-hook-form + Yup antes de enviar
- Error 401 → mensaje en campo contraseña, no toast
- Token guardado en localStorage clave fuelroute_token
- Redirige a /mapa tras login exitoso

2. Pantalla de Inicio de Sesión



Descripción:

Permite a los usuarios autenticarse en el sistema mediante:

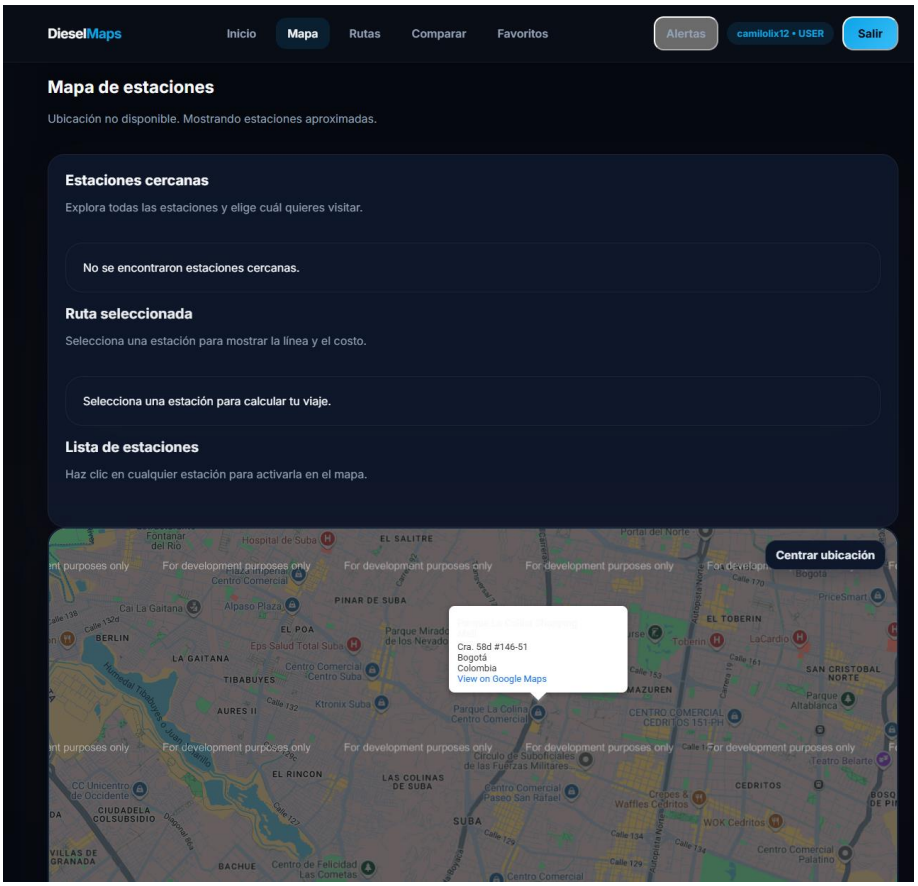
- Correo electrónico
- Contraseña

Incluye botón “Iniciar Sesión” y enlace a “Registrarse”.

Funcionalidad:

- Password: min 8 chars, mayúscula + minúscula + número
- Error 409 email → error en campo email específico
- Error 409 username → error en campo username

3. Pantalla Principal – Mapa Interactivo



Descripción:

Es la pantalla central del sistema. Contiene:

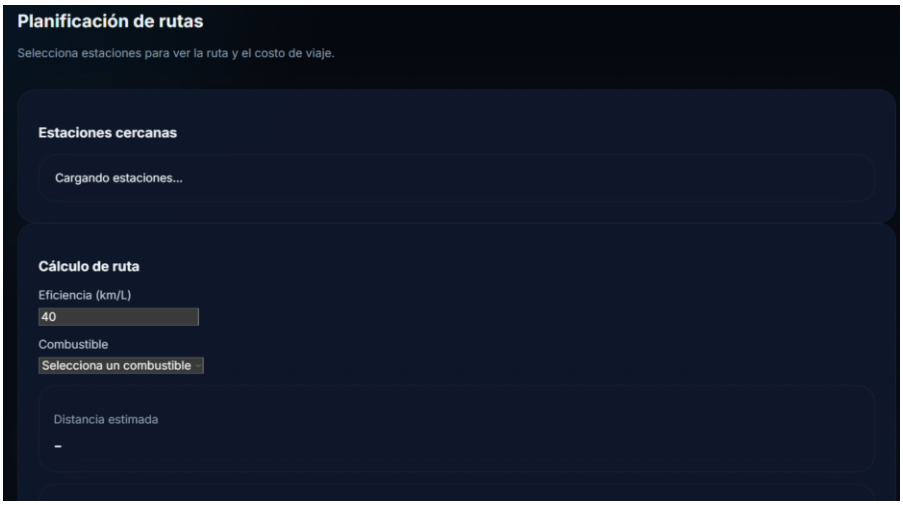
- Mapa interactivo (Google Maps o API similar).
- Estaciones cercanas
- Ruta seleccionada
- Lista de estaciones
- Menú de navegación superior.

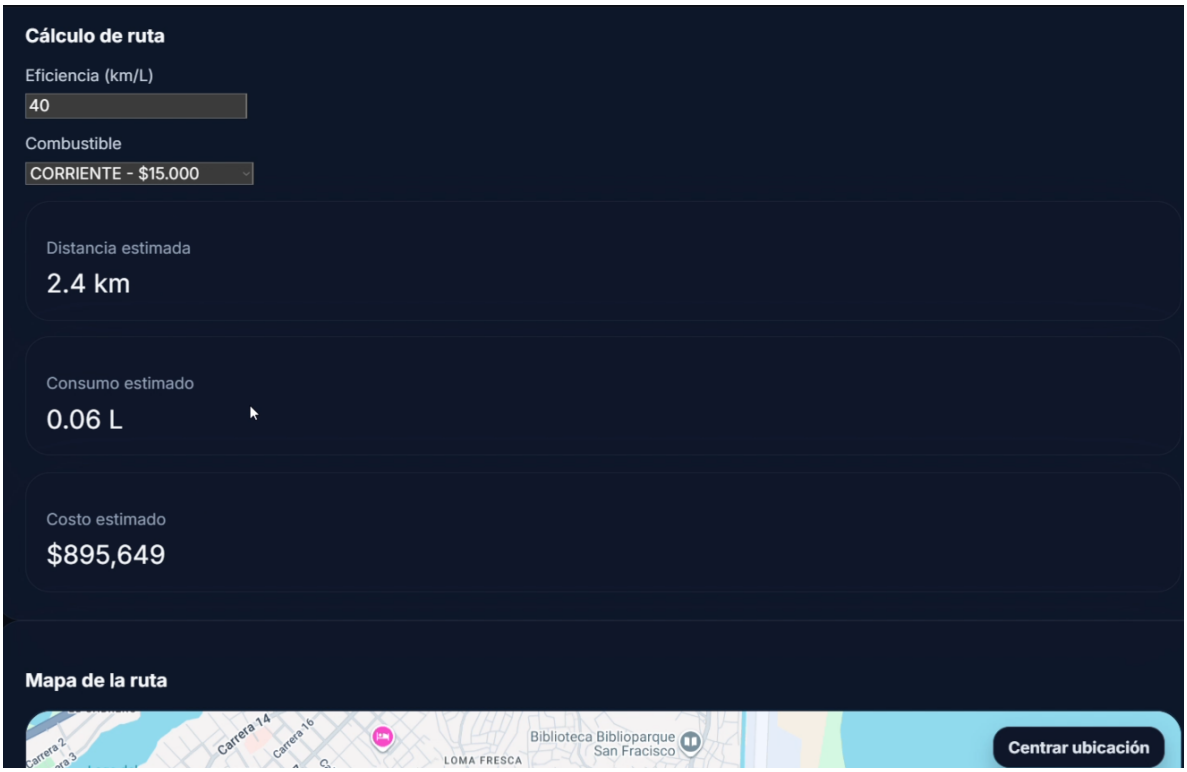
Funcionalidad:

- Visualización de ubicaciones registradas.

- Mostrar marcadores de estaciones de combustible.
- Mostrar detalles al hacer clic en un marcador.
- Geolocalización en tiempo real del usuario.

4. Pantalla de Rutas





Descripción:

Muestra la información detallada de una estación seleccionada.

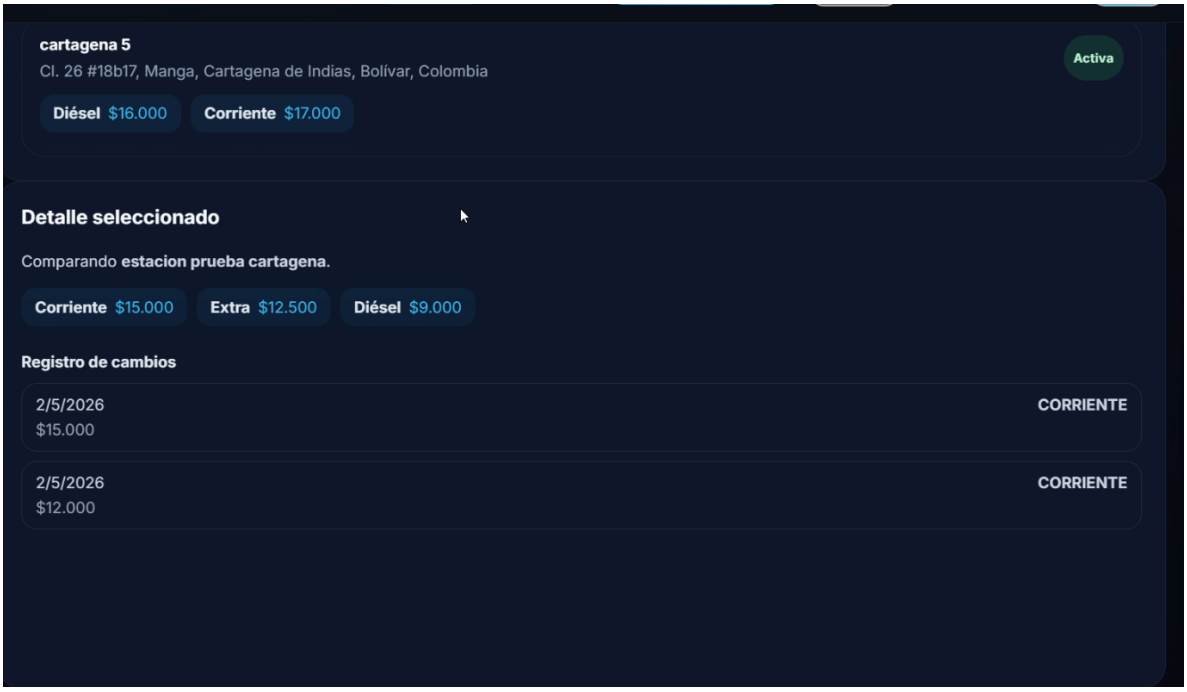
Información mostrada:

- Cálculo de ruta
- Seleccionar tipo de combustible
- Seleccionar eficiencia
- Distancia estimada
- Consumo estimado
- Costo estimado
- Mapa de ruta

Funcionalidad:

- Mostrar detalles almacenados en base de datos.
- Calcular distancia desde la ubicación actual.
- Permitir reportar actualización de precios (usuario registrado).

5. Pantalla para Comparar



Descripción:

Revisa hasta tres estaciones cercanas y visualiza sus precios históricos

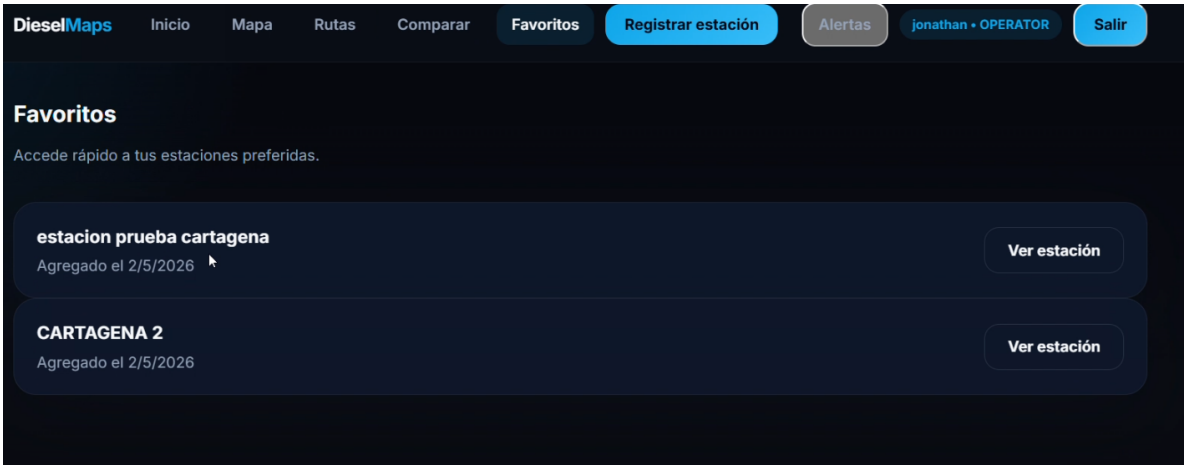
- Punto de origen
- Punto de destino
- Consumo del vehículo (km/l)
- Precio del combustible

Incluye botón “Comparar”.

Funcionalidad:

- Cálculo automático del consumo estimado.
- Mostrar costo total del viaje.
- Mostrar distancia total.
- Tiempo estimado de recorrido.

6. Pantalla de Favoritos



Descripción:

Accede rápido a tus estaciones preferidas.

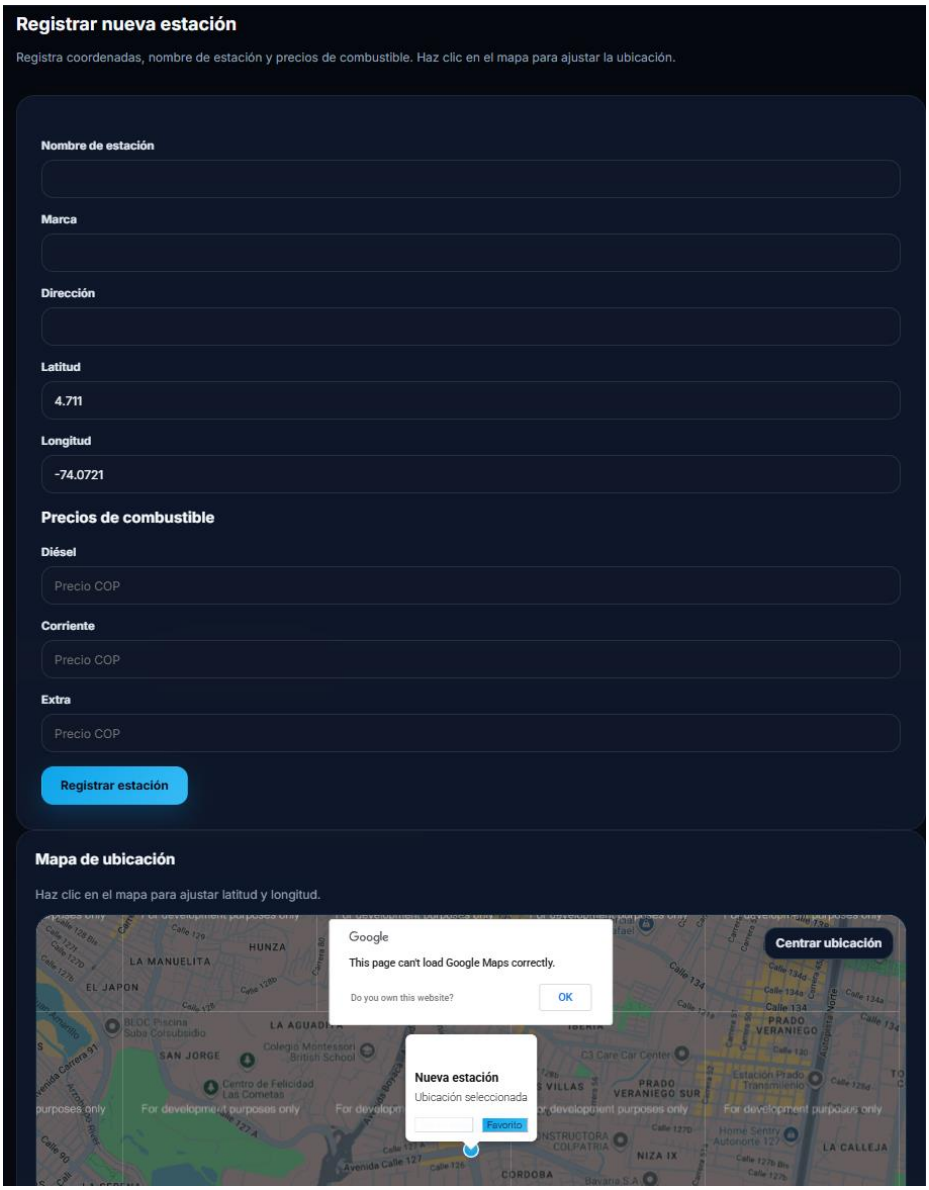
Incluye:

- Estaciones

Funcionalidad:

Se muestran las estaciones guardadas previamente

7) Registrar estación



Descripción:

Sirve para registrar una nueva estación con sus coordenadas, nombre de estación y precios de combustible y ajustar la ubicación.

Funcionalidad:

Guardar los datos en la base de datos

- **Flujo de Navegación**

1. Usuario ingresa al sistema.
2. Se registra o inicia sesión.
3. Accede al mapa principal.
4. Busca estación.
5. Consulta detalles.
6. Puede simular costo o calcular ruta.
7. Si es administrador, accede al panel de gestión.

- **Reglas de Validación**

- Correo con formato válido.
- Contraseña mínimo 8 caracteres.
- Solo administradores pueden registrar estaciones.
- Campos obligatorios no pueden quedar vacíos.
- Precios deben ser valores numéricos positivos.

- **Diseño**

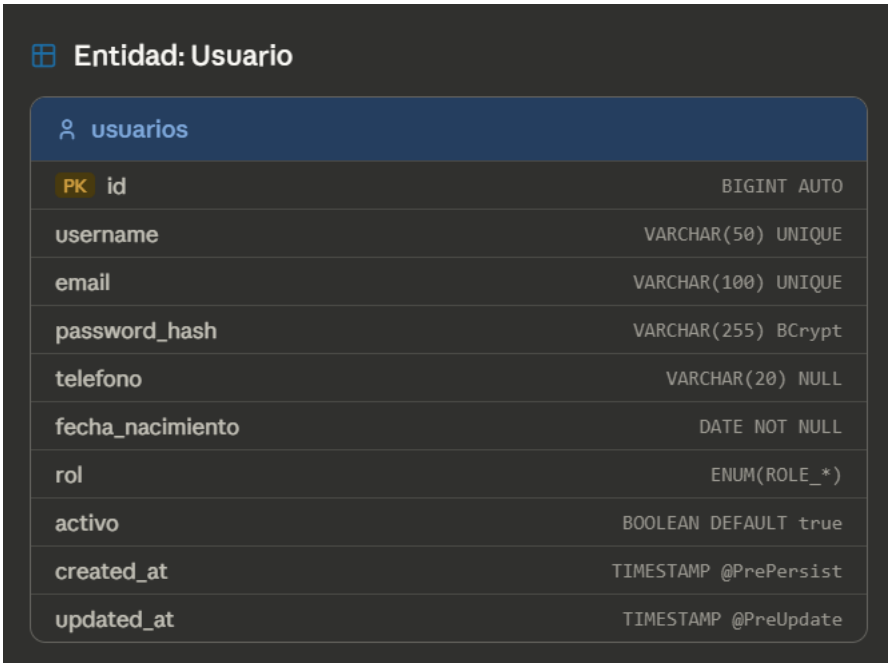
El sistema será adaptable a:

- Computadores.
- Tablets.

- Dispositivos móviles.

4.2 Modelo de Datos (Entidad-Relación)

Entidades



Entidad: Estacion

estaciones	
PK id	BIGINT
nombre	VARCHAR(100)
marca	VARCHAR(80)
direccion	VARCHAR(255)
ciudad	VARCHAR(100) IDX
departamento	VARCHAR(100)
latitud	DECIMAL(10,8) IDX
longitud	DECIMAL(11,8) IDX
horario	VARCHAR(100)
activo	BOOLEAN softDel
created_at / updated_at	TIMESTAMP

Entidad: TipoCombustible

tipos_combustible	
PK id	BIGINT
nombre	VARCHAR(50) UNIQUE
descripcion	VARCHAR(200)
activo	BOOLEAN

Datos iniciales: Corriente · Extra · Diesel · Gas

Entidad: Ruta

rutas	
PK id	BIGINT
FK usuario_id	→ usuarios
origen_nombre / lat / lng	DECIMAL(10,8)
destino_nombre / lat / lng	DECIMAL(10,8)
distancia_km	DECIMAL(10,2)
consumo_litros	DECIMAL(10,3)
costo_total_cop	DECIMAL(12,2)
ahorro_cop	DECIMAL(12,2)
FK tipo_combustible_id	→ tipos
rendimiento_kmpl	DECIMAL(5,2)
created_at	TIMESTAMP

Entidad: PrecioHistorial

$ precios_historial — append-only (nunca UPDATE)

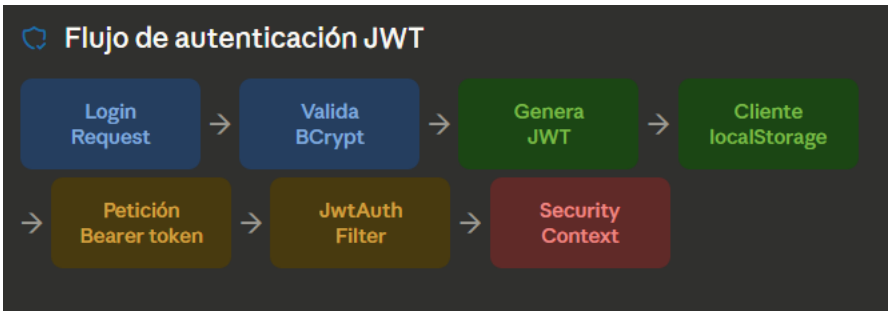
PK id	BIGINT	FK estacion_id	→ estaciones	FK tipo_com	
precio_cop	DECIMAL(10,2)	FK registrado_por	→ usuarios	fecha_registr	

Precio actual = `ORDER BY fecha_registro DESC LIMIT 1`. El historial completo se conserva para análisis de tendencias.

Relaciones entre entidades

Origen	Cardinalidad	Destino	Fetch	Regla de negocio
Usuario	1 → N	Ruta	LAZY	CASCADE DELETE. Usuario ve solo sus propias rutas.
Usuario	1 → N	PrecioHistorial	LAZY	Auditoría: quién registró cada precio.
Estacion	1 → N	PrecioHistorial	LAZY	RESTRICT DELETE. Datos históricos preservados.
TipoCombustible	1 → N	PrecioHistorial	LAZY	RESTRICT DELETE. Un tipo agrupa todos sus precios.
TipoCombustible	1 → N	Ruta	LAZY	Cada ruta calcula costo para un solo tipo de combustible.

Flujo de autenticación JWT



Matriz de roles y permisos

Operación	USUARIO	OPERADOR	ADMIN
Ver estaciones / buscar cercanas	✓	✓	✓
Calcular ruta óptima	✓	✓	✓
Ver historial propio	✓	✓	✓
Crear / editar estaciones	✗	✓	✓
Registrar precios	✗	✓	✓
Soft delete de estaciones	✗	✗	✓
Gestionar usuarios y roles	✗	✗	✓

La autorización se aplica con `@PreAuthorize` a nivel de **servicio**, no solo en el controlador. Esto garantiza que ninguna invocación interna pueda saltar la validación de rol.

Credenciales

Credencial	Variable de entorno	Restricción
Clave JWT	`JWT_SECRET`	Mínimo 256 bits en Base64. Generado con `openssl rand -base64 64`
Google Maps API Key (backend)	`GOOGLE_MAPS_API_KEY`	Restringida por IP del servidor. Habilita solo Directions API
Google Maps API Key (frontend)	`VITE_GOOGLE_MAPS_API_KEY`	Restringida por HTTP referrer. Habilita Maps JavaScript API

4.3 Modelo de Clases (UML)

Clases Principales

- **Clase Usuario**

Atributos

- id_usuario : int
- nombre: String
- correo: String
- contraseña: String
- rol: String

Métodos

- registrar ()
- autenticar ()
- actualizarDatos()
- eliminar ()

- **Clase Estación**

Atributos

- id_estacion : int

- nombre: String
- dirección: String
- latitud: double
- longitud: double
- marca: String

Métodos

- registrarEstacion()
- actualizarEstacion()
- eliminarEstacion()

- **Clase Precio Combustible**

Atributos

- id_precio : int
- tipo_combustible : String
- precio: double
- fecha_actualizacion : Date

Métodos

- actualizarPrecio()
- consultarPrecio()

- Clase SimulacionViaje

Atributos

- id_simulacion : int
- origen: String
- destino: String
- distancia_km : double
- costo_estimado : double

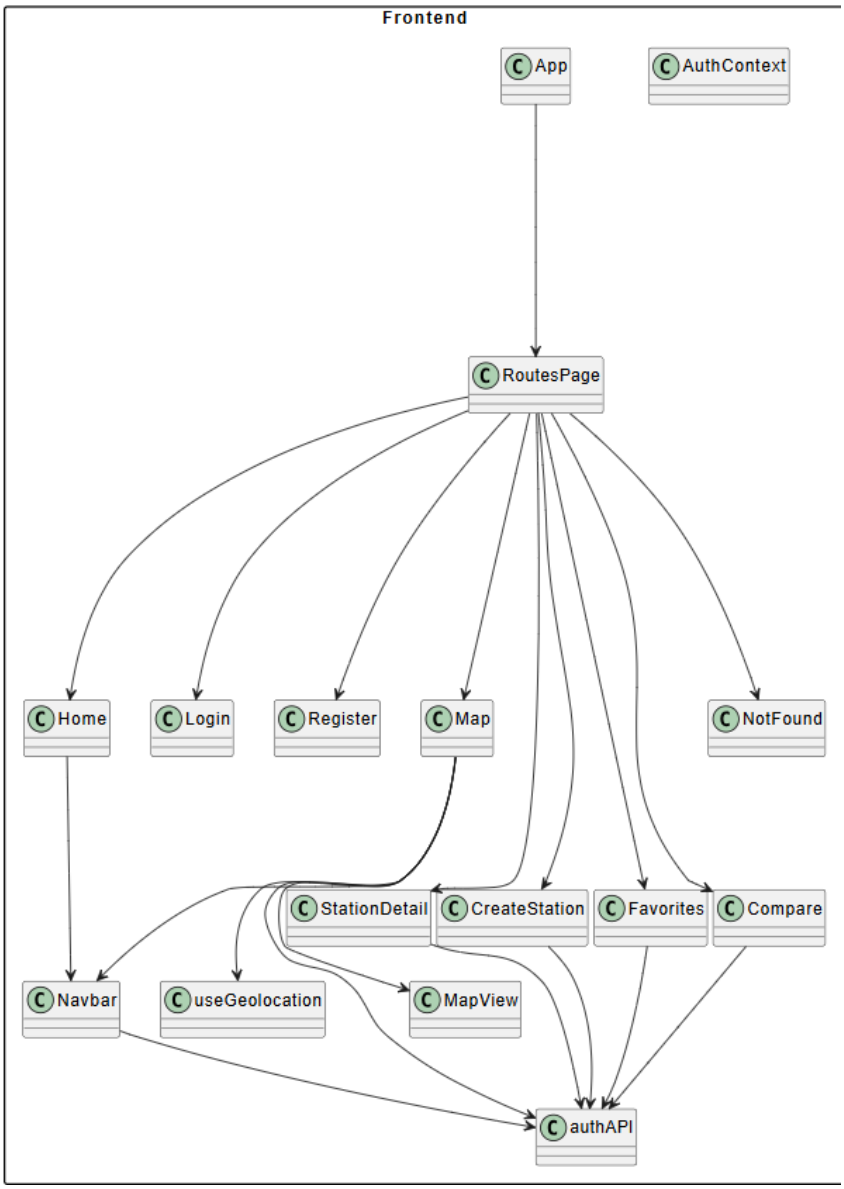
Métodos

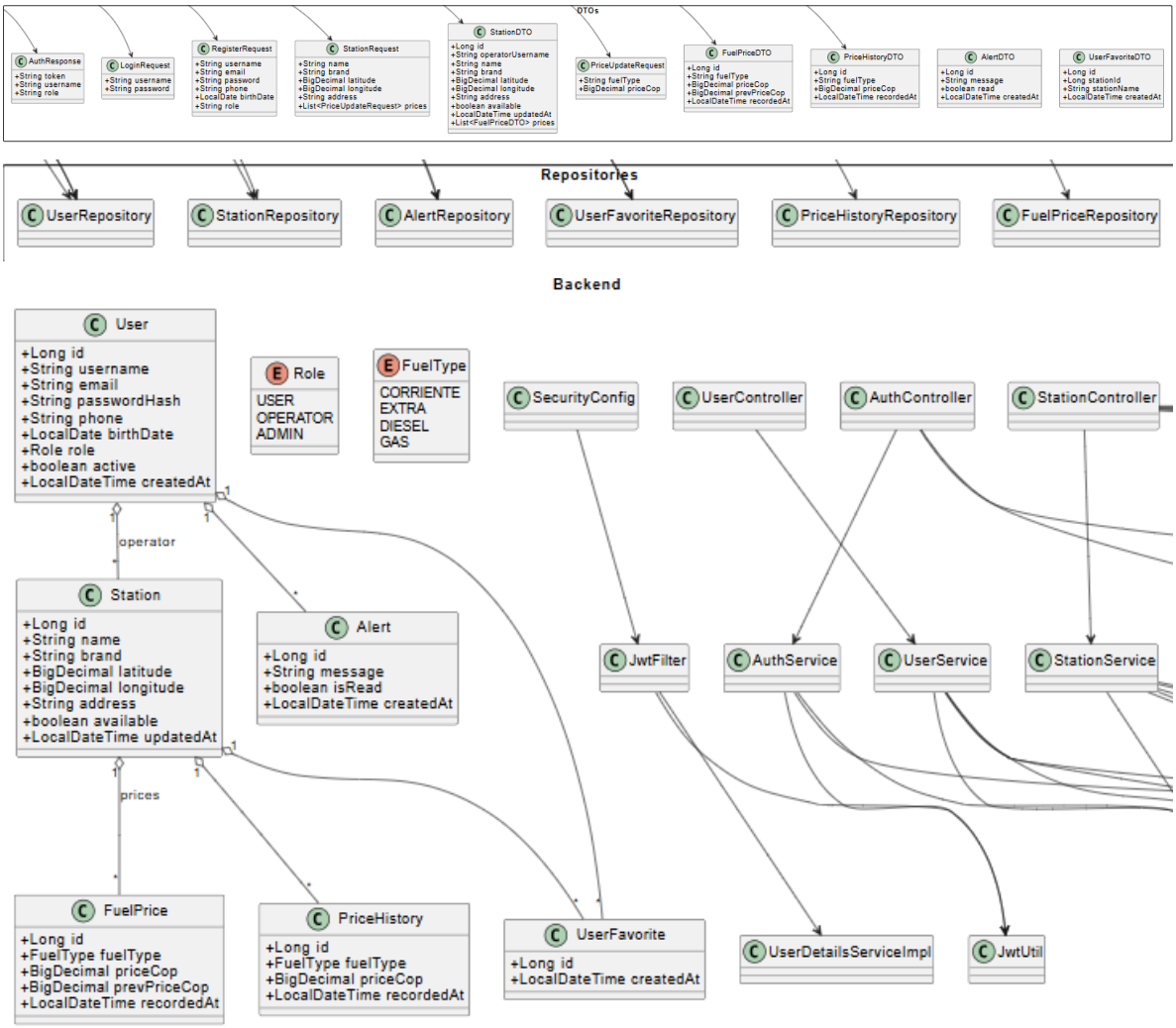
- calcularDistancia()
- calcularCosto()
- guardarSimulacion()

- Relaciones (herencia, agregación, composición)

- Usuario 1 — N SimulacionViaje
- Estacion 1 — N PrecioCombustible
- Existe asociación entre clases.
- Sistema usa todas las clases.

- Diagrama UML de clases




DTOs
AuthResponse
+String token
+String username
+String role
LoginRequest
+String username
+String password
RegisterRequest
+String username
+String email
+String password
+String phone
+LocalDate birthDate
+String role
StationRequest
+String name
+String brand
+BigDecimal latitude
+BigDecimal longitude
+String address
+List<PriceUpdateRequest> prices
StationDTO
+Long id
+String operatorUsername
+String name
+String brand
+BigDecimal latitude
+BigDecimal longitude
+String address
+boolean available
+LocalDateTime updatedAt
+List<FuelPriceDTO> prices
PriceUpdateRequest
+String fuelType
+BigDecimal priceCop
FuelPriceDTO
+Long id
+String fuelType
+BigDecimal priceCop
+BigDecimal prevPriceCop
+LocalDateTime recordedAt
PriceHistoryDTO
+Long id
+String fuelType
+BigDecimal priceCop
+LocalDateTime recordedAt
AlertDTO
+Long id
+String message
+boolean read
+LocalDateTime createdAt
UserFavoriteDTO
+Long id
+Long stationId
+String stationName
+LocalDateTime createdAt
Repositories
UserRepository
StationRepository
AlertRepository
UserFavoriteRepository
PriceHistoryRepository
FuelPriceRepository
Backend
User
+Long id
+String username
+String email
+String passwordHash
+String phone
+LocalDate birthDate
+Role role
+boolean active
+LocalDateTime createdAt
Role
USER
OPERATOR
ADMIN
FuelType
CORRIENTE
EXTRA
DIESEL
GAS
SecurityConfig
UserController
AuthController
StationController
operator
Station
+Long id
+String name
+String brand
+BigDecimal latitude
+BigDecimal longitude
+String address
+boolean available
+LocalDateTime updatedAt
Alert
+Long id
+String message
+boolean isRead
+LocalDateTime createdAt
JwtFilter
AuthService
UserService
StationService
prices
FuelPrice
+Long id
+FuelType fuelType
+BigDecimal priceCop
+BigDecimal prevPriceCop
+LocalDateTime recordedAt
PriceHistory
+Long id
+FuelType fuelType
+BigDecimal priceCop
+LocalDateTime recordedAt
UserFavorite
+Long id
+LocalDateTime createdAt
UserDetailsServiceImpl
JwtUtil